

AS - BUILT DOCUMENTATION

티켓 예매 서비스 인프라 설계 대비 실제 구현 기록

최초 설계안과 실제 구축 결과를 비교하고, 서버별 최종 명령어·방화벽 규칙·보안 조치·실행한 테스트 스크립트를 하나로 정리한 최종 산출 문서다.

네트워크 172.16.60.0/24 · 서버 8대 · 작성일 2026.07.06 · 최시은

1. 설계 대비 변경사항

최초 설계 문서와 실제 구축 결과를 대조했다. 특히 이상탐지(IDS) 배치 위치는 실제로 검증하며 설계를 수정한 대표적인 사례다.

항목	최초 계획	실제 구현	변경 이유
네트워크 대역	신규 VMnet10, 10.10.0.0/24 (홈랩과 완전 분리)	172.16.60.0/24 (기존 홈랩과 같은 대역 재사용, 서버/계정은 전부 신규)	실습 효율을 위해 기존 네트워크 재사용, 서버 단위 분리 유지
이상탐지(IDS) 배치	별도 monitor 서버(트래픽 경로 밖)에 Suricata+ELK	실제 트래픽이 지나가는 queue-gate에 Suricata 직접 설치 (ELK는 보류)	별도 서버는 패킷 자체를 볼 수 없어 탐지 불가 — 실습으로 확인 후 구조 변경
부하테스트 실행 위치	k6-runner 전용 VM	맥북(Homebrew k6)에서 직접 실행	VM 리소스 절약, 실제 사용자와 더 유사한 외부 경로에서 테스트
회원 인증(로그인/MFA)	bcrypt 해시, 90일 만료, 관리자 TOTP MFA 설계 완료	프론트엔드 화면만 존재, 백엔드 미구현	좌석예약 동시성 검증을 우선순위로 진행, 다음 단계 과제로 이관
관리자 접근	개념 설계만 존재	맥북 ~/.ssh/config에 서버별 Host 등록, ProxyJump로 실제 운영	반복 접속 편의성 확보
서버 대수	8대 (bastion·db·db-replica·redis·was×2·queue-gate·web-lb)	동일 8대로 완성	계획대로 진행

가장 중요한 설계 변경 — "IDS는 무엇을 탐지하느냐"보다 "어디에 배치하느냐"가 먼저라는 것을 monitor 서버 실패 사례로 직접 확인했다.

2. 최종 서버 IP 매핑

서버	IP	역할
bastion	172.16.60.142	SSH 유일 진입점
db-01	172.16.60.143	MariaDB Primary (쓰기)
redis	172.16.60.144	좌석 재고 원자연산
was-01	172.16.60.145	예약 API #1
queue-gate	172.16.60.146	대기열/Rate limiting + Suricata IDS
web-lb	172.16.60.147	로드밸런서 + 정적 페이지 서빙
db-01-replica	172.16.60.148	MariaDB Replica (읽기 전용)
was-02	172.16.60.151	예약 API #2

3. 서버별 최종 구성 및 명령어

3.1 bastion (.142)

```
sudo apt update && sudo apt install -y fail2ban ufw openssh-server
sudo sed -i 's/#PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sed -i 's/#PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo systemctl restart ssh
sudo ufw default deny incoming
sudo ufw allow from <관리자_고정IP> to any port 22 proto tcp
sudo ufw enable
sudo systemctl enable --now fail2ban
```

3.2 db-01 (.143)

```
sudo apt install -y mariadb-server
sudo mariadb-secure-installation

sudo sed -i "s/bind-address.*/bind-address          = 172.16.60.143/" /etc/mysql/mariadb.conf.d/50-server.cnf
sudo tee /etc/mysql/mariadb.conf.d/60-replication.cnf > /dev/null << 'EOF'
[mariadb]
server-id = 1
log_bin = /var/log/mysql/mariadb-bin
binlog_format = ROW
EOF
sudo systemctl restart mariadb
```

```
CREATE DATABASE ticket;
CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  game_id INT NOT NULL, seat_id VARCHAR(10) NOT NULL, buyer_name VARCHAR(100),
  created_at DATETIME DEFAULT NOW(),
  UNIQUE KEY unique_seat (game_id, seat_id)
);
CREATE USER 'was_writer'@'172.16.60.%' IDENTIFIED BY '<비밀번호>';
GRANT SELECT, INSERT, UPDATE ON ticket.* TO 'was_writer'@'172.16.60.%';
CREATE USER 'replicator'@'172.16.60.148' IDENTIFIED BY '<비밀번호>';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'172.16.60.148';
SHOW MASTER STATUS;
```

```
sudo ufw default deny incoming
sudo ufw allow from 172.16.60.145 to any port 3306 proto tcp
sudo ufw allow from 172.16.60.151 to any port 3306 proto tcp
sudo ufw allow from 172.16.60.148 to any port 3306 proto tcp
sudo ufw allow from 172.16.60.142 to any port 22 proto tcp
sudo ufw enable
```

3.3 db-01-replica (.148)

```
sudo apt install -y mariadb-server
mysqldump -h 172.16.60.143 -u was_writer -p<pw> \
  --single-transaction --skip-lock-tables --databases ticket > ticket-dump.sql
sudo mariadb -e "CREATE DATABASE IF NOT EXISTS ticket;"
sudo mariadb ticket < ticket-dump.sql
```

```
sudo tee /etc/mysql/mariadb.conf.d/60-replication.cnf > /dev/null << 'EOF'
[mariadb]
server-id = 2
read_only = 1
bind-address = 172.16.60.148
EOF
sudo systemctl restart mariadb
```

```
CHANGE MASTER TO
  MASTER_HOST='172.16.60.143', MASTER_USER='replicator', MASTER_PASSWORD='<비밀번호>',
  MASTER_LOG_FILE='mariadb-bin.000001', MASTER_LOG_POS=804;
START SLAVE;
-- 계정 재생성 이력이 binlog에 남아 SQL_Running:No 발생 시:
SET GLOBAL sql_slave_skip_counter = 1;
START SLAVE;
```

```
sudo ufw default deny incoming
sudo ufw allow from 172.16.60.145 to any port 3306 proto tcp
sudo ufw allow from 172.16.60.151 to any port 3306 proto tcp
sudo ufw allow from 172.16.60.142 to any port 22 proto tcp
sudo ufw enable
```

3.4 redis (.144)

```
sudo apt install -y redis-server
sudo sed -i "s/^bind 127.0.0.1.*bind 172.16.60.144/" /etc/redis/redis.conf
sudo sed -i "s/^# requirepass.*requirepass <비밀번호>/" /etc/redis/redis.conf
sudo systemctl restart redis-server
```

```
sudo ufw default deny incoming
sudo ufw allow from 172.16.60.145 to any port 6379 proto tcp
sudo ufw allow from 172.16.60.151 to any port 6379 proto tcp
sudo ufw allow from 172.16.60.142 to any port 22 proto tcp
sudo ufw enable
```

3.5 was-01 (.145) / was-02 (.151)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

```
mkdir ~/ticket-api && cd ~/ticket-api
npm init -y
npm install express mysql2 redis dotenv
```

```
cat > .env << 'EOF'
DB_HOST=172.16.60.143
DB_USER=was_writer
DB_PASSWORD=<비밀번호>
DB_NAME=ticket
REDIS_HOST=172.16.60.144
REDIS_PASSWORD=<비밀번호>
PORT=3000
EOF
```

```
sudo ufw default deny incoming
sudo ufw allow from 172.16.60.147 to any port 3000 proto tcp
sudo ufw allow from 172.16.60.142 to any port 22 proto tcp
sudo ufw enable

node server.js
```

3.6 queue-gate (.146) — 대기열 + Suricata

```
sudo apt install -y nginx
sudo rm /etc/nginx/sites-enabled/default
```

```
sudo ufw default deny incoming
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow from 172.16.60.142 to any port 22 proto tcp
sudo ufw enable
```

```
# /etc/nginx/conf.d/queue.conf
limit_req_zone $binary_remote_addr zone=global:20m rate=10r/s;
limit_conn_zone $binary_remote_addr zone=addr:10m;
server {
    listen 80;
    limit_req zone=global burst=20 nodelay;
    limit_conn addr 5;
    location / { proxy_pass http://172.16.60.147; }
}
```

```
sudo add-apt-repository universe
sudo add-apt-repository ppa:oisf/suricata-stable
sudo apt update && sudo apt install -y suricata
```

```
sudo sed -i 's/HOME_NET:./HOME_NET: "[172.16.60.0/24]"/' /etc/suricata/suricata.yaml
sudo sed -i 's/eth0/ens160/g' /etc/suricata/suricata.yaml
sudo sed -i 's/ KiB/kb/g; s/ MiB/mb/g' /etc/suricata/suricata.yaml
```

```
sudo tee /var/lib/suricata/rules/suricata.rules > /dev/null << 'EOF'
alert http any any -> any any (msg:"Excessive booking API calls detected";
  http.uri; content:"/api/book-seat";
  threshold: type threshold, track by_src, count 20, seconds 10;
  sid:1000001; rev:1;)
EOF
sudo systemctl enable --now suricata
```

3.7 web-lb (.147)

```
sudo apt install -y nginx
sudo rm /etc/nginx/sites-enabled/default
```

```
sudo ufw default deny incoming
sudo ufw allow from 172.16.60.146 to any port 80 proto tcp
sudo ufw allow from 172.16.60.146 to any port 443 proto tcp
sudo ufw allow from 172.16.60.142 to any port 22 proto tcp
sudo ufw enable
```

```
# /etc/nginx/conf.d/upstream.conf
upstream was_pool {
    least_conn;
    server 172.16.60.145:3000 max_fails=3 fail_timeout=10s;
    server 172.16.60.151:3000 max_fails=3 fail_timeout=10s;
}
server {
    listen 80;
    root /var/www/ticket;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
    location /api/ { proxy_pass http://was_pool/api/; }
}
```

4. 방화벽 최종 매트릭스

서버	열린 포트	허용 출발지	용도
bastion	22	관리자 고정 IP	SSH 진입점
queue-gate	80, 443	전체(공개)	사용자 진입점
queue-gate	22	bastion	관리용
web-lb	80, 443	queue-gate	대기열 통과 트래픽만
was-01/02	3000	web-lb	API 수신
redis	6379	was-01, was-02	좌석 원자연산
db-01	3306	was-01, was-02, db-replica	쓰기 + 복제
db-01-replica	3306	was-01, was-02	읽기 전용
모든 서버 22번은 bastion(172.16.60.142)에서만 예외적으로 허용			

5. 보안 규칙 요약

규칙	구현 내용	상태
단일 진입점(Bastion)	모든 SSH 접근은 bastion 경유만 허용	완료/검증
ufw 화이트리스트	전 서버 default-deny + 출발지 IP별 허용	완료/nmap 검증
DB 계정 최소권한	was_writer(쓰기), replicator(복제 전용) 분리	완료
Redis 인증/바인딩	requirepass, 내부 IP 바인딩	완료
좌석 동시성 제어	Redis SETNX 원자연산으로 중복예약 차단	완료/K6 검증
트래픽 폭증 방어	Rate limiting(초당 10건, burst 20)	완료/K6 검증
실시간 이상탐지	Suricata 룰 기반 반복요청 탐지	완료/실전 탐지 성공
보안 헤더	CSP, Referrer-Policy, Permissions-Policy 적용	3/5 적용 (HSTS·X-Content-Type-Options 남음)
회원 인증(bcrypt/MFA)	비밀번호 해시, 90일 만료, 관리자 TOTP	설계만 완료, 백엔드 미구현

정직하게 남겨야 할 부분 — 회원 인증은 아직 실제 DB에 계정이 저장되고 bcrypt로 해시되는 걸 구현/검증하지 못했다. 지금까지 검증한 것은 가용성(트래픽폭증)·무결성(동시성) 중심이라, 기밀성(개인정보 보호) 축을 채우려면 회원가입 API 구현이 다음 우선순위다.

6. 실행한 스크립트 및 시나리오

6.1 예약 API 핵심부 (server.js)

```
app.post('/api/book-seat', async (req, res) => {
  const { gameId, seatId, buyerName } = req.body;
  const lockKey = `seat:${gameId}:${seatId}`;
  const acquired = await redisClient.set(lockKey, buyerName, { NX: true });
  if (!acquired) return res.status(409).json({ message: '이미 선택된 좌석입니다' });
  await db.query('INSERT INTO bookings (game_id, seat_id, buyer_name) VALUES (?, ?, ?)',
    [gameId, seatId, buyerName]);
  res.status(200).json({ message: '예약 성공', server: SERVER_NAME, gameId, seatId, buyerName });
});
```

6.2 시나리오 3 — 좌석 동시클릭 (seat-race.js)

```
import http from 'k6/http';
import { check } from 'k6';
export const options = {
  scenarios: { same_seat_attack: {
    executor: 'shared-iterations', vus: 100, iterations: 100, maxDuration: '10s',
  }}
};
export default function () {
  const payload = JSON.stringify({ gameId: 1, seatId: 'A7', buyerName: `user-${__VU}` });
  const res = http.post('http://172.16.60.145:3000/api/book-seat', payload, {
    headers: { 'Content-Type': 'application/json' },
  });
  check(res, { '200 또는 409 응답': (r) => [200, 409].includes(r.status) });
}
```

결과: 내부/외부 네트워크 등 총 3가지 조건에서 DB에 정확히 1건만 저장됨을 확인.

6.3 시나리오 2 — 트래픽 폭증 대기열 비교 (traffic-spike.js)

```
import http from 'k6/http';
export const options = {
  stages: [
    { duration: '10s', target: 50 },
    { duration: '5s', target: 300 },
    { duration: '20s', target: 300 },
    { duration: '10s', target: 0 },
  ],
};
export default function () {
  http.get('http://172.16.60.146/api/health');
}
```

지표	대기열 OFF	대기열 ON
checks_failed	1.27%	0.00%
http_req_duration max	60,000ms	108.8ms

6.4 Suricata 실전 탐지 테스트 (attack-sim.js)

```
import http from 'k6/http';
export const options = { vus: 10, duration: '15s' };
export default function () {
  http.post('http://172.16.60.146/api/book-seat', JSON.stringify({
    gameId: 1, seatId: 'A5', buyerName: 'attacker'
  })), { headers: { 'Content-Type': 'application/json' } });
}
```

결과: queue-gate의 fast.log에 "Excessive booking API calls detected" 알림 다수 발생 확인.

6.5 자체 모의해킹 — nmap / Nikto

```
nmap -Pn -p 22,80,443,3000,3306,6379 172.16.60.147
nmap -Pn -p 22,80,443,3000,3306,6379 172.16.60.145
```



```
nmap -Pn -p 22,80,443,3000,3306,6379 172.16.60.146
nikto -h http://172.16.60.146
```

결과: queue-gate만 80 open, 나머지 서버는 외부(맥북) 기준 전 포트 filtered — 설계와 실재가 일치. Nikto에서 보안헤더 5종 누락 발견, 3종 조치 완료.

본 문서는 2026년 7월 6일 진행한 구축·검증 세션의 최종 as-built 기록이며, 비밀번호 등 민감값은 실제 값 대신 플레이스홀더로 표기했다.